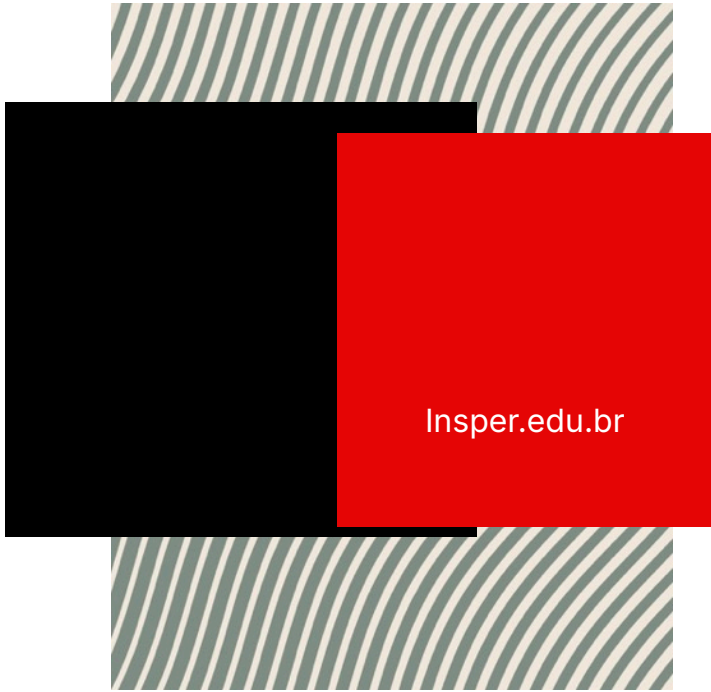


Clínica II

Modelos de linguagem

Uma coluna nova a partir do texto das sentenças

Insper

A decorative graphic on the right side of the slide. It features a black square overlapping a red square, which is placed on top of a light gray square with dark gray wavy lines. The URL 'Insper.edu.br' is written in white on the red square.

Insper.edu.br

Um trabalho com cliente

Um trabalho com cliente

Diferente de outros trabalhos finais, este tem um cliente.

Com o que o grupo precisa se preocupar aqui que num trabalho final comum não precisaria?

Um trabalho com cliente

RESPOSTA

preocupação

o que o cliente pergunta

custo

quanto vou pagar para rodar a base?

velocidade

em quanto tempo sai o resultado?

experiência de quem usa

quem vai usar consegue rodar e entender?

custo-efetividade

o resultado vale o que custa?

Tudo isso começa em duas decisões: usar um modelo de linguagem ou não e, se sim, **qual**.

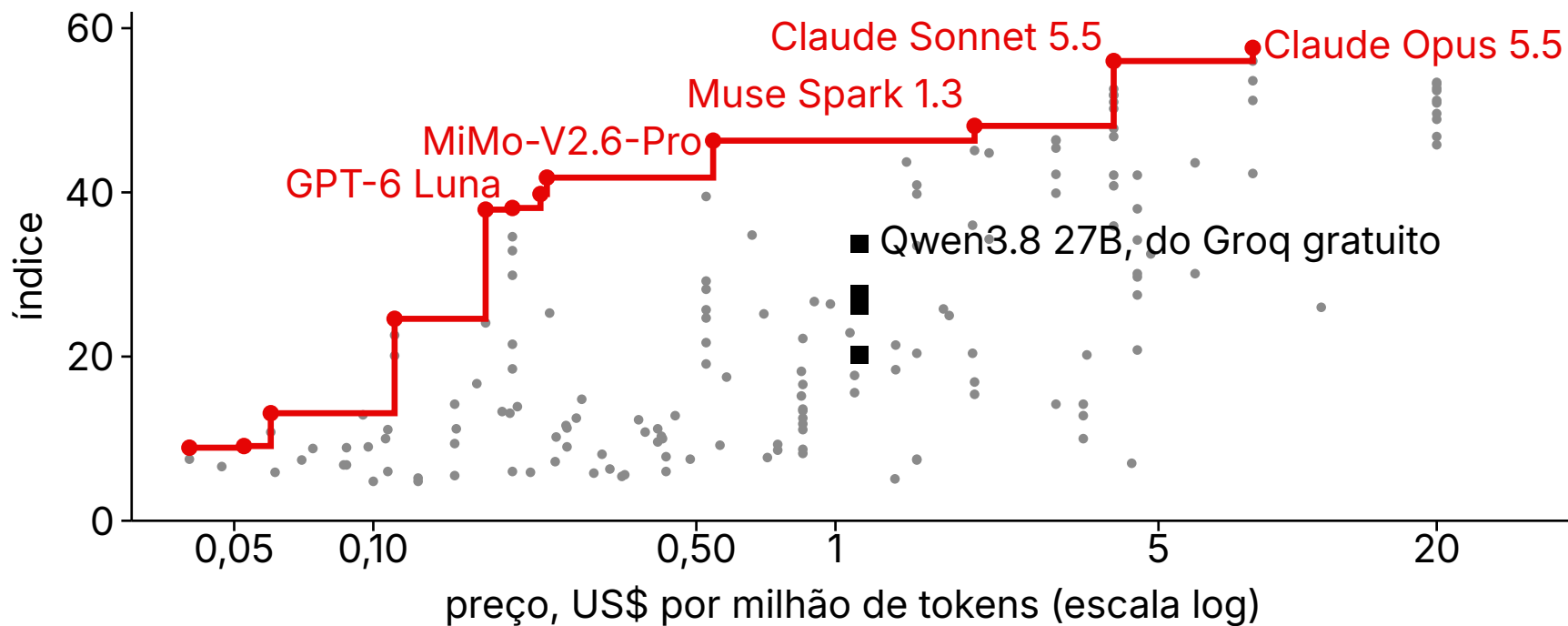
Onde comparar modelos

O Artificial Analysis (artificialanalysis.ai) testa os modelos por conta própria e publica três medidas:

medida	como sai
inteligência	um índice de 0 a 100, a partir de baterias de testes
velocidade	tokens por segundo na resposta
preço	dólares por milhão de tokens

O índice mede testes gerais, e não a tarefa do grupo. Só testar numa amostra diz se o modelo **serve para o cliente**.

Inteligência contra preço



Cada ponto é um dos 169 modelos com preço, em 05/10/2026. **Em vermelho, a fronteira.** Os quadrados pretos são o modelo da demonstração, do menor ao maior raciocínio.

A fronteira de Pareto

Um modelo está na fronteira quando **nenhum outro mais barato é mais inteligente**. Fora dela, sempre há um modelo melhor pelo mesmo preço, ou mais barato com a mesma inteligência.

na fronteira	índice	US\$ por milhão
MiMo-V2.6-Flash	37,9	0,18
GPT-6 Luna	38,1	0,20
GLM 5.3 Flash	41,8	0,24
MiMo-V2.6-Pro	46,3	0,54
Muse Spark 1.3	48,1	2,00
Claude Sonnet 5.5	56,0	4,00
Claude Opus 5.5	57,6	8,00

É a fronteira de **05/10/2026**. Sai modelo novo toda semana: consulte o site no dia da escolha.

Onde rodar

onde	o que dá de graça
Groq, plano gratuito	sem cartão; 1.000 pedidos por dia; é o da demonstração
Google Cloud	US\$ 300 por 90 dias, com cartão
AWS	US\$ 100, mais até US\$ 100 por tarefas, por 12 meses
Azure for Students	US\$ 100 por 12 meses, sem cartão

Nas nuvens, os créditos pagam modelos de vários fornecedores. Se o cliente exigir que os dados fiquem no Brasil, pergunte ao provedor, modelo a modelo: a região de São Paulo existe, mas **nem todo modelo roda nela**.

A coluna que falta

22.866 sentenças e uma pergunta

Um exemplo: uma base com 22.866 sentenças do TJSP em ações contra bancos, disponibilizadas entre 2024 e 2026. O cliente quer saber **em quantas o pedido foi negado, e por quê**.

As colunas da base

identificação	onde	o quê
cd_processo, id_processo	comarca, foro, vara	classe, assunto
magistrado	data_disponibilizacao	decisao, o texto inteiro

Ler as 22.866 sentenças

Nenhuma coluna diz o resultado: ele está no texto de decisao. Uma sentença da base tem, em média, 1.189 palavras.

Quantas horas o grupo levaria para ler todas, a 200 palavras por minuto?

Ler as 22.866 sentenças

RESPOSTA

```
palavras = 22866 * 1189  
horas = palavras / 200 / 60  
print(round(horas))
```

2266

São 283 dias de oito horas, sem pausa para anotar. Ler tudo à mão não cabe no semestre: o grupo precisa de um programa que leia por ele.

Procurar a palavra

Marcar as sentenças procedentes

```
import re
dispositivos = [
    "JULGO IMPROCEDENTES os pedidos da exordial.",
    "JULGO PROCEDENTE EM PARTE o processo",
    "julgo procedentes os pedidos da parte requerente",
    "JULGO EXTINTO o processo, sem resolução de mérito",
]
for texto in dispositivos:
    print(bool(re.search("procedente", texto, re.I)))
```

O grupo marca como procedente a sentença em que a palavra aparece. Preveja as quatro linhas e diga, para cada uma, se a marcação está certa.

Marcar as sentenças procedentes

RESPOSTA

True
True
True
False

A primeira é **improcedência**: "IMPROCEDENTES" contém "procedente". A segunda é procedência em parte, contada como procedência inteira. Só a terceira e a quarta estão certas.

A correção da colega

```
import re
dispositivos = [
    "JULGO IMPROCEDENTES os pedidos da exordial.",
    "JULGO PROCEDENTE EM PARTE o processo",
    "julgo procedentes os pedidos da parte requerente",
    "JULGO EXTINTO o processo, sem resolução de mérito",
]
for texto in dispositivos:
    print(bool(re.search(r"\bprocedente", texto, re.I)))
```

O `\b` exige que a palavra comece ali. Preveja de novo: o que ainda sai errado?

A correção da colega

RESPOSTA

False

True

True

False

"IMPROCEDENTES" saiu, mas a procedência **em parte** continua contada como procedência. Cada conserto fecha um buraco e deixa outro.

A palavra na base inteira

o texto contém	sentenças
"procedente"	12.454
"improcedente"	7.348
"parcialmente procedente" ou "procedente em parte"	4.136
"julgo procedente"	2.505

E há "JULGO PROCEDENTE, em parte", "homologo a desistência", que não é acordo, e "JULGO IMPROCEDENTE o pedido de indenização por danos morais" numa sentença que concedeu os danos materiais.

Por que o pedido foi negado

Em muitas dessas ações, o autor diz que nunca contratou o empréstimo. Um motivo frequente de improcedência é o juiz concluir que **o banco provou a contratação**. Três trechos com esse motivo:

"A prova produzida revela que a assinatura aposta no contrato partiu do punho subscritor da parte autora."

"Fê-lo, por certo, porque tomou os empréstimos, recebeu o dinheiro em conta e com isso anuiu."

"A contratação eletrônica, mediante uso de senha pessoal e intransferível [...] confirma a existência e validade do negócio jurídico."

Escreva um padrão que ache, nas 22.866, as sentenças com esse motivo.

Por que o pedido foi negado

RESPOSTA

Os três trechos dizem a mesma coisa sem uma palavra em comum. "Assinatura", "contrato" ou "senha" também aparecem em sentenças em que o banco **não** provou nada.

O padrão procura uma forma escrita. O motivo é um sentido, e cada juiz o escreve de um jeito.

Regex ou modelo de linguagem

	regex	modelo de linguagem
serve para	formato fixo: número CNJ, valor em R\$, inscrição na OAB	sentido: o motivo, a tese, se houve fraude
custo	nenhum, roda no computador	cobra por token e tem limite de uso
como erra	no texto que foge do padrão	com respostas plausíveis e erradas

Pedir ao modelo

Uma pergunta ao modelo pelo código

```
from langchain_groq import ChatGroq

modelo = ChatGroq(model="qwen/qwen3.8-27b")
texto = "JULGO IMPROCEDENTES os pedidos da exordial."
resposta = modelo.invoke("Qual foi o resultado? " + texto)
print(resposta.content)
```

O modelo roda nos computadores da empresa que o oferece. O programa manda o texto pela internet e recebe a resposta. A empresa precisa saber de quem é a conta: a **chave de API**.

O que acontece numa chamada

passo

- 1 o programa monta o pedido: o modelo, o texto e a chave
- 2 manda o pedido pela internet aos servidores do Groq
- 3 o Groq confere a chave, roda o modelo e conta os tokens
- 4 devolve a resposta e a contagem de tokens
- 5 lança o uso na conta dona da chave: cobrança, ou o limite do plano gratuito

A chave é a única coisa no pedido que diz **de quem é a conta**.

A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário código.

O que é um .env

Um arquivo de texto na pasta do projeto, com o que não pode ir no código. O ChatGroq acha a chave no ambiente.

o arquivo .env

```
GROQ_API_KEY=gsk_...
```

o notebook

```
from dotenv import load_dotenv
load_dotenv() # o .env vai para o ambiente
modelo = ChatGroq(model="qwen/qwen3.8-27b")
```

O notebook vai para a banca; o .env **fica na máquina**. No Colab, a chave vai em "Secrets", o ícone de chave.

A chave no ambiente

Animação no navegador: [animacao-api-llm.html](#), cenário ambiente.

A sugestão do colega

O grupo vai publicar o notebook num repositório público para a banca conferir. Um colega propõe deixar a chave do Groq escrita numa célula, para a banca rodar sem configurar nada, e apagar a linha depois da banca.

Qual o problema nisso?

A sugestão do colega

RESPOSTA

Quem abrir o arquivo enquanto ele estiver no ar usa a chave, e o uso cai na conta do grupo: no plano pago, a cobrança; no gratuito, o limite diário, que acaba antes do trabalho.

Apagar depois não desfaz: o histórico do repositório e as cópias guardam a versão com a chave. A chave exposta se **revoga** no painel do Groq, e a banca usa a própria chave, lida do ambiente.

No notebook: seções 1 a 3

Instalar, criar a chave do Groq, guardá-la no .env e fazer a primeira pergunta ao modelo.

Quanto custa

Mandar a base inteira

A base tem 177.873.339 caracteres. Cada 4 caracteres valem cerca de 1 token, e o preço é por milhão de tokens.

modelo	entrada, por milhão de tokens
A	US\$ 5,00
B	US\$ 0,15

Quanto custa mandar cada sentença uma vez, contando só a entrada, em cada modelo?

Mandar a base inteira

RESPOSTA

```
tokens = 177873339 / 4  
print(round(tokens / 1_000_000 * 5.00, 2))  
print(round(tokens / 1_000_000 * 0.15, 2))
```

222.34

6.67

São 44,5 milhões de tokens. O modelo A custa 33 vezes o B, e a resposta de cada sentença ainda se paga à parte.

O limite do plano gratuito

plano gratuito do Groq, um modelo	limite
tokens por dia	200 mil
tokens por minuto	8 mil
sentenças por dia, na média da base	cerca de 100
dias para a base inteira	222

E 275 sentenças passam de 8 mil tokens sozinhas: nem cabem num minuto do plano gratuito.

Reduzir antes de mandar

etapa	sentenças	tokens
a base como chegou	22.866	44,5 milhões
sem textos repetidos	20.013	38,6 milhões
só o assunto que a pergunta pede	menos	menos
amostra para testar o prompt	50	cerca de 100 mil

A base repete 2.853 textos inteiros. Testar o prompt numa amostra custa centavos e acha o erro antes da conta grande.

O que são tokens de raciocínio

Muitos modelos escrevem um rascunho antes de responder: releem o pedido, testam uma leitura, conferem. O rascunho também é feito de tokens, e **se paga como saída**.

o dispositivo

"JULGO IMPROCEDENTE a pretensão [...]. Em relação ao pedido formulado no item "c" [...], o feito fica extinto sem resolução de mérito"

rascunho do modelo, 203 tokens (trecho)

"The main ruling is "JULGO IMPROCEDENTE a pretensão" [...]. There's a secondary part about item "c" [...], but the main result of the sentence is "improcedente.""

resposta

improcedente

Dois jeitos de gastar computação

	no treino	no uso
o que cresce	o modelo, os dados, o tempo de treino	o rascunho antes de cada resposta
quem paga	o fornecedor, uma vez	quem usa, a cada pergunta
exemplo	gpt-oss de 20 para 120 bilhões de parâmetros: índice 9,0 → 11,6	Qwen3.8 27B sem raciocínio e no máximo: índice 20 → 34

Os dois melhoram a resposta. O do uso **entra na conta do cliente**, a cada sentença.

Os tokens de raciocínio

Um modelo cobra US\$ 0,15 por milhão de tokens de entrada e US\$ 0,60 por milhão de saída. Por sentença: 2.000 tokens de entrada, 100 de resposta e 1.500 de raciocínio.

Quanto custam 20.000 sentenças?

Os tokens de raciocínio

RESPOSTA

```
entrada = 2000 * 0.15 / 1_000_000
saida = (100 + 1500) * 0.60 / 1_000_000
print(round(20000 * (entrada + saida), 2))
```

25.2

O raciocínio se cobra como **saída**. Quem conta só a resposta chega a US\$ 7,20, um terço do valor real.

No notebook: seções 4 e 5

Carregar a base, tirar as repetidas, calcular o custo e sortear uma amostra que caiba no plano gratuito.

Do texto à coluna

Cinco respostas do modelo

Respostas reais de um modelo a "Qual foi o resultado desta sentença?"

1. "O resultado desta sentença foi totalmente desfavorável para quem entrou com a ação (o autor). [...]"
2. "O resultado dessa sentença foi favorável à pessoa que entrou com a ação (autor), embora tenha sido classificada como "procedente em parte" [...]"
3. "O resultado foi a extinção do processo sem que o juiz julgasse quem está certo ou errado [...]"
4. "O resultado desta sentença foi o encerramento (extinção) do processo, porque o autor [...] desistiu [...]"
5. "O resultado foi a vitória da autora [...]. O juiz julgou o pedido procedente [...]"

Quantas são procedentes? Que código contaria isso nas 22.866 respostas?

Cinco respostas do modelo

RESPOSTA

Uma, a quinta. A primeira é improcedência e abre por "desfavorável"; a segunda é procedência em parte e abre por "favorável".

O modelo entendeu as cinco, mas respondeu cada uma de um jeito. Contar essas respostas exige de novo procurar palavras: o **problema da regex voltou**.

Uma tabela entra, uma tabela sai

O `dataframeit` manda cada linha de uma tabela a um modelo e devolve a mesma tabela com colunas novas. Ele precisa de quatro coisas:

o quê	para quê
a tabela	uma linha por sentença; <code>text_column</code> diz qual coluna tem o texto
uma classe	cada campo vira uma coluna; a <code>description</code> diz ao modelo o que coletar
um prompt	a instrução; o texto da linha entra no lugar de <code>{texto}</code>
<code>provider</code> e <code>model</code>	onde roda e qual modelo responde

Descrever o que coletar

```
from typing import Literal
from pydantic import BaseModel, Field
class Decisao(BaseModel):
    resultado: Literal[
        "procedente", "parcialmente procedente",
        "improcedente", "extinto sem resolução do mérito",
        "homologação", "outro"] = Field(
        description="O resultado da sentença.")
```

Motivos e má-fé na mesma classe

Os dois campos seguintes da classe `Decisao`, depois do resultado

```
motivos: list[Literal[
    "contratação comprovada", "encargos sem abusividade",
    "golpe por culpa da vítima ou de terceiro",
    "falta de prova mínima", "prescrição ou decadência",
    "outro"]] = Field(description="Por que o pedido "
                                "foi negado.")
ma_fe: bool = Field(description="A sentença fala em "
                                "litigância de má-fé?")
```

Três tipos de campo

tipo	o modelo é obrigado a
<code>Literal[...]</code>	escolher exatamente uma das opções listadas
<code>list[Literal[...]]</code>	escolher nenhuma, uma ou várias das opções
<code>bool</code>	responder True ou False

O tipo fixa o formato da resposta. A description diz ao modelo **o que coletar** em cada campo.

Uma chamada para a tabela inteira

```
from dataframeit import dataframeit

# sentencas: o DataFrame da base, uma linha por sentença
# Decisao: a classe dos slides anteriores
tabela = dataframeit(sentencas, Decisao,
                     "Classifique a sentença: {texto}",
                     provider="groq", text_column="decisao",
                     model="qwen/qwen3.8-27b")

print(tabela["resultado"].value_counts())
```

Em cada linha, o texto da coluna decisao entra no lugar de {texto}, e o que volta preenche as colunas da classe.

O que volta

na tabela devolvida	o que é
<code>resultado, motivos, ma_fe</code>	uma coluna por campo da classe
<code>_dataframeit_status</code>	aparece se alguma linha falhou, e diz qual
<code>_error_details</code>	o motivo da falha, linha a linha

Se o limite do plano gratuito interromper, rodar **a mesma chamada** de novo processa só as linhas que faltam.

Três sentenças viram três linhas

Animação no navegador: [animacao-extracao.html](#), cenário sentencas.

A classe da outra pergunta

O cliente quer saber também se a sentença reconheceu fraude na contratação e de quanto foi a indenização por dano moral, quando houve.

Escreva a classe com esses dois campos.

A classe da outra pergunta

RESPOSTA

```
class Indenizacao(BaseModel):  
    fraude_reconhecida: bool = Field(  
        description="Reconheceu fraude na contratação?")  
    valor_dano_moral: float | None = Field(  
        description="Dano moral em reais; vazio se não  
houve")
```

O valor é número, e não texto, para a coluna poder ser somada. Aceitar None deixa o campo **vazio** quando não houve dano moral, em vez de obrigar o modelo a inventar um zero.

No notebook: seções 6 e 7

Escrever a classe com as descriptions e rodar o dataframeit na amostra.

O modelo acertou?

Conferir cinco anotações

Leia o trecho e confira o que o modelo anotou

trecho	resultado	ma_fe
"HOMOLOGO o pedido de desistência e JULGO EXTINTO O PROCESSO SEM RESOLUÇÃO DO MÉRITO [...]"	homologação	False
"JULGO PARCIALMENTE PROCEDENTE a ação para: DECLARAR a inexigibilidade do débito [...]"	parcialmente procedente	False
"JULGO PROCEDENTE o pedido para [...] condenar os réus [...] ao pagamento de R\$ 15.000,00 [...]. Ainda JULGO IMPROCEDENTE o pedido de indenização por dano moral."	improcedente	False
"Finalmente, não há que se falar em litigância de má-fé. [...] JULGO IMPROCEDENTE o pedido inicial."	improcedente	True
"JULGO IMPROCEDENTE a ação em relação ao [banco]. [...] Em relação à [corrê], [...] julga-se extinto o feito sem resolução de mérito"	extinto sem resolução do mérito	False

Conferir cinco anotações

RESPOSTA

Só a segunda está certa, e nenhum erro é ao acaso:

- | | |
|---|--|
| 1 | "homologo" puxou para homologação, mas desistência é extinção sem resolução do mérito |
| 3 | um pedido acolhido e outro negado: parcialmente procedente |
| 4 | a sentença cita a má-fé para afastá-la: ma_fe é False |
| 5 | a extinção é só quanto à corrê; contra o banco, improcedente |

A classe não dizia **o que conta** como cada opção.

Sortear, ler e comparar

Animação no navegador: [animacao-validade.html](#), cenário amostra.

Melhorar a descrição

```
resultado: Literal[..., "acordo homologado", "outro"] = Field(
    description="Resultado do pedido principal. Acordo
    homologado "
    "é só acordo entre as partes; desistência homologada
    é "
    "extinção. Um pedido acolhido e outro negado é
    parcial.")
ma_fe: bool = Field(
    description="True só se condenou o autor por má-fé; "
    "citar para afastar é False.")
```

Depois, rodar de novo e conferir numa **amostra nova**.

Quantas sentenças ler

Animação no navegador: [animacao-amostragem.html](#).

O que escrever no relatório

Com a descrição melhorada, o grupo sorteou outras 50 das 20.013 sentenças, leu cada uma e comparou com o modelo: 46 resultados iguais.

Escreva o que o relatório pode afirmar sobre as 20.013.

O que escrever no relatório

RESPOSTA

Que o modelo concordou com a leitura humana em 46 de 50 sentenças sorteadas: 92%, com margem de uns **7,5 pontos** para mais ou para menos. O número estima a base inteira sem garanti-la, porque as outras 19.963 ninguém leu.

Cada divergência se relê: o erro pode ser do modelo ou de quem leu. Se os erros do modelo tiverem padrão, o relatório diz qual, para quem usar a tabela saber onde desconfiar.

No notebook: seções 8 e 9

Ler o dispositivo de cada sentença da amostra, comparar com o modelo e guardar a tabela.